

■ Java Interview

Mastery Guide

Director & Senior Level Deep Dive

OOP • SOLID • Collections • Streams • JVM • Design Patterns • Git • JUnit • Spring Boot

Every Question + Deep Answer + Generated Follow-up Questions

Chapter 1: SOLID Principles

The Foundation of Scalable Object-Oriented Design

SOLID is an acronym coined by Robert C. Martin ('Uncle Bob'). These 5 principles are the backbone of maintainable, extensible, and testable software. At the director/architect level, you're expected to not just name them — but explain the failure modes when they're violated, and how they relate to design patterns, microservices, and system architecture.

Q1. Explain the Single Responsibility Principle (SRP). What is 'a reason to change'? How does it relate to cohesion?

Definition: A class should have only ONE reason to change — meaning it should have only ONE job or responsibility.

Deep Explanation: SRP is fundamentally about COHESION. High cohesion means elements of a class belong together. When a class handles multiple concerns (e.g., business logic + DB + reporting), any change to any of those concerns forces you to modify the same class — increasing the risk of regression bugs.

The 'Reason to Change' means: Who is the STAKEHOLDER requesting the change? If multiple departments (HR, Finance, IT) can independently request changes to the same class — it violates SRP.

```
// ■ VIOLATION – EmployeeService has 3 reasons to change:
// 1. HR changes salary rules
// 2. DBA changes persistence logic
// 3. Finance changes report format
class EmployeeService {
    void calculateSalary(Employee e) { /* HR logic */ }
    void saveEmployee(Employee e) { /* DB logic */ }
    void generatePayslip(Employee e) { /* report logic */ }
}

// ■ CORRECT – Each class has one owner/stakeholder
class SalaryCalculator { double calculate(Employee e) { return e.baseSalary * 1.2; } }
class EmployeeRepository { void save(Employee e) { /* only DB */ } }
class PayslipGenerator { void generate(Employee e) { /* only reports */ } }
```

✓ SRP → High Cohesion → Fewer regression bugs → Easier testing → Classes become reusable units.

■ **Note:** At senior level: SRP at class level. At architect level: SRP applies to MICROSERVICES too — each service owns ONE business domain.

■ **Director-Level Follow-up Questions:**

- How does SRP relate to Conway's Law in microservice architecture?
- If a class has 2 methods but they change for different reasons, is SRP violated even though it's small?
- How would you refactor a 5000-line God Class that violates SRP without breaking existing functionality?
- Explain the difference between SRP and separation of concerns (SoC).

Q2. Explain the Open/Closed Principle. Why is 'open for extension, closed for modification' critical in production systems?

Definition: Software entities (classes, modules, functions) should be OPEN for extension but CLOSED for modification.

Why Critical in Production: When you modify existing code, you risk breaking functionality that already passed QA, UAT, and is live. OCP lets you add features by ADDING new code, never touching proven code. This is the principle behind plugin architectures, strategy patterns, and feature flags.

```
// ■ VIOLATION – Every new discount type requires modifying PricingEngine
class PricingEngine {
    double calculate(Order order) {
        if (order.type == 'SEASONAL') return order.price * 0.8;
        if (order.type == 'LOYALTY') return order.price * 0.85;
        if (order.type == 'EMPLOYEE') return order.price * 0.5;
        // Add new type? MODIFY this class! ■
        return order.price;
    }
}

// ■ OCP via Strategy Pattern
interface DiscountStrategy { double apply(double price); }

class SeasonalDiscount implements DiscountStrategy {
    public double apply(double price) { return price * 0.8; }
}

class LoyaltyDiscount implements DiscountStrategy {
    public double apply(double price) { return price * 0.85; }
}

class PricingEngine {
```

```
private DiscountStrategy strategy;
PricingEngine(DiscountStrategy s) { this.strategy = s; }
double calculate(Order order) { return strategy.apply(order.price); }
}
// Adding 'FLASH_SALE' discount? Just create new class! Touch ZERO existing code.
```

✓ OCP is achieved through abstraction (interfaces/abstract classes) + polymorphism. The Strategy, Decorator, and Template Method patterns are all implementations of OCP.

■ **Note:** OCP + Dependency Injection = Spring Boot's core philosophy. @Autowired injects implementations without modifying consuming classes.

■ Director-Level Follow-up Questions:

- Can OCP be violated in a GOOD way? When is modifying existing code acceptable?
- How does OCP relate to the Open API specification in REST API versioning?
- Explain how Abstract Factory pattern achieves OCP at the object-creation level.
- In Agile/iterative development, code constantly changes. How do you practically apply OCP?

Q3. Explain Liskov Substitution Principle with a real example. What does 'behavioral subtyping' mean?

Definition (Barbara Liskov, 1987): If S is a subtype of T, then objects of type T may be replaced with objects of type S without altering the correctness of the program.

Behavioral Subtyping means: A subclass must honor the CONTRACT of the parent — preconditions cannot be strengthened, postconditions cannot be weakened, invariants must be preserved.

```
// CLASSIC VIOLATION – Square/Rectangle problem
class Rectangle {
protected int width, height;
public void setWidth(int w) { this.width = w; }
public void setHeight(int h) { this.height = h; }
public int area() { return width * height; }
}

class Square extends Rectangle {
@Override public void setWidth(int w) { width = height = w; } // BREAKS CONTRACT!
@Override public void setHeight(int h) { width = height = h; } // BREAKS CONTRACT!
}

// Client code – works for Rectangle but BREAKS for Square:
void testArea(Rectangle r) {
r.setWidth(5); r.setHeight(10);
// Expected: 50, but Square gives 100! Program is INCORRECT.
```

```

assert r.area() == 50; // FAILS for Square!
}

// ■ FIX – Don't use inheritance for IS-A if behavior differs
interface Shape { int area(); }
class Rectangle implements Shape {
    int w, h;
    Rectangle(int w, int h) { this.w=w; this.h=h; }
    public int area() { return w * h; }
}
class Square implements Shape {
    int side;
    Square(int side) { this.side = side; }
    public int area() { return side * side; }
}

// REAL-WORLD: Java's Stack extends Vector – LSP VIOLATION!
// Stack has push/pop semantics; Vector allows arbitrary insertion.
// Stack<Integer> s = new Stack<>();
// s.add(0, 99); // Valid for Vector but BREAKS Stack semantics!

```

✓ LSP violation detection: If you write 'instanceof' checks in calling code to handle a subtype differently — you've violated LSP.

■ **Warning:** Java's own `java.util.Stack` extends `Vector` — a famous LSP violation in the standard library!

■ Director-Level Follow-up Questions:

- How do you detect LSP violations programmatically in a large codebase?
- Explain how LSP relates to the 'Tell, Don't Ask' principle.
- How does LSP apply to interface implementations, not just class inheritance?
- What is the relationship between LSP and Design by Contract (DbC)?

Q4. Explain Interface Segregation Principle. How does it prevent 'fat interfaces' and reduce coupling?

Definition: No client should be forced to depend on methods it does not use. Split large interfaces into smaller, role-specific interfaces.

```

// ■ FAT INTERFACE – Robot is forced to implement biological methods
interface Worker {
    void work();
    void eat(); // Robot can't eat!
    void sleep(); // Robot doesn't sleep!
}

```

```

void getBonus(); // Robots don't get bonuses!
}

class Robot implements Worker {
public void work() { /* ok */ }
public void eat() { throw new UnsupportedOperationException(); }
public void sleep() { throw new UnsupportedOperationException(); }
public void getBonus() { throw new UnsupportedOperationException(); }
}

// ■ SEGREGATED – Each interface has one clear role
interface Workable { void work(); }
interface Feedable { void eat(); }
interface Restable { void sleep(); }
interface Compensatable { void getBonus(); }

class Human implements Workable, Feedable, Restable, Compensatable { ... }
class Robot implements Workable { public void work() { /* only this */ } }

// REAL-WORLD EXAMPLE – JPA Repository interfaces:
// CrudRepository → basic CRUD
// PagingAndSortingRepository → extends Crud + adds paging
// JpaRepository → extends Paging + adds JPA-specific methods
// You inject only what you need – ISP in practice!

```

✓ ISP reduces the 'footprint' of dependencies. Smaller interfaces = easier mocking in tests = better testability.

■ **Note:** *ISP in REST APIs: Don't return a massive DTO with 50 fields when the client needs only 3. Use projections/DTOs — this is ISP at the API design level.*

■ Director-Level Follow-up Questions:

- How does ISP differ from SRP? Can you violate one without violating the other?
- How does ISP relate to GraphQL vs REST API design philosophy?
- In Spring Data JPA, how does the Repository hierarchy demonstrate ISP?

Q5. Explain Dependency Inversion Principle. How is it different from Dependency Injection?

Definition: 1) High-level modules should not depend on low-level modules. Both should depend on abstractions. 2) Abstractions should not depend on details. Details should depend on abstractions.

DIP vs DI: DIP is a PRINCIPLE (what to aim for). Dependency Injection (DI) is a TECHNIQUE (mechanism) to achieve DIP. Spring's @Autowired is DI. The fact that you inject an interface rather than a concrete class

is DIP.

```
// ■ VIOLATION – High-level OrderService depends on low-level MySQL detail
class MySQLOrderRepository {
void save(Order o) { /* MySQL-specific SQL */ }
}

class OrderService {
private MySQLOrderRepository repo = new MySQLOrderRepository(); // HARD dependency!
void placeOrder(Order o) {
// validate...
repo.save(o);
// Can NEVER switch to PostgreSQL/MongoDB without changing OrderService!
}
}

// ■ DIP – Both depend on abstraction (interface)
interface OrderRepository {
void save(Order o);
Optional<Order> findById(Long id);
}

class MySQLOrderRepository implements OrderRepository {
public void save(Order o) { /* MySQL impl */ }
public Optional<Order> findById(Long id) { /* MySQL query */ }
}

class MongoOrderRepository implements OrderRepository {
public void save(Order o) { /* Mongo impl */ }
public Optional<Order> findById(Long id) { /* Mongo query */ }
}

// High-level module depends ONLY on interface
@Service
class OrderService {
private final OrderRepository repo; // interface, not impl
@Autowired
OrderService(OrderRepository repo) { this.repo = repo; } // DI injects it
void placeOrder(Order o) { repo.save(o); }
}

// Switch DB? Just register different bean – OrderService untouched!
```

✓ DIP enables TESTABILITY: inject a mock OrderRepository in unit tests. It enables FLEXIBILITY: swap implementations without changing business logic.

■ **Note:** The 'D' in SOLID is perhaps the most important for Spring Boot developers. @Autowired, @Component, @Service all implement DIP mechanically.

■ **Director-Level Follow-up Questions:**

- How does DIP enable parallel development in large teams?
 - What is the Dependency Inversion Container (IoC Container) and how does Spring implement it?
 - When should you NOT use DIP? Is there overhead to always abstracting?
 - How does DIP relate to Hexagonal Architecture (Ports and Adapters)?
-

Chapter 2: OOP — Deep Concepts

Abstraction, Encapsulation, Inheritance, Polymorphism

Q6. What is abstraction vs encapsulation? Developers often confuse them — explain with concrete code.

Abstraction: WHAT an object does — hiding complexity, exposing only essential behavior. Implemented via abstract classes and interfaces.

Encapsulation: HOW data is protected — binding data and methods, restricting direct field access. Implemented via access modifiers (private/protected).

```
// ABSTRACTION — you know WHAT a car does, not HOW the engine works
abstract class Vehicle {
    abstract void start(); // WHAT: must start somehow
    abstract void stop(); // WHAT: must stop somehow
    void fuelCheck() { System.out.println('Checking fuel...'); } // common behavior
}

class ElectricCar extends Vehicle {
    public void start() { System.out.println('Silent electric start'); }
    public void stop() { System.out.println('Regenerative braking'); }
}

// ENCAPSULATION — HOW data is hidden and protected
class BankAccount {
    private double balance; // HIDDEN — no direct access
    private String accountNumber; // HIDDEN
    private List<String> transactions = new ArrayList<>();

    public void deposit(double amount) {
        if (amount <= 0) throw new IllegalArgumentException('Amount must be positive');
        balance += amount;
        transactions.add('DEPOSIT: +' + amount);
    }

    public double getBalance() { return balance; } // controlled read
    // NO setBalance() — external code cannot set arbitrary balance!
```

```

}

// 'Escaping Reference' problem (encapsulation breach):
class Employee {
    private List<String> skills = new ArrayList<>();
    public List<String> getSkills() { return skills; } // BUG! returns mutable ref!
    // External code: emp.getSkills().clear() – bypasses encapsulation!

    // FIX: return unmodifiable view
    public List<String> getSkillsSafe() { return Collections.unmodifiableList(skills); }
}

```

✓ Abstraction = design-time concern (interfaces/contracts). Encapsulation = runtime concern (data protection). You can have one without the other.

■ Director-Level Follow-up Questions:

- *What is the 'escaping reference' problem? How does it breach encapsulation?*
- *How do Records in Java 16+ handle encapsulation? Are they truly encapsulated?*
- *Can an interface provide abstraction without encapsulation?*

Q7. Explain protected access modifier. What are its pitfalls in large codebases?

Protected: Accessible within the SAME PACKAGE and by SUBCLASSES (even in different packages).

```

package com.company.base;
public class Animal {
    protected String species; // accessible by subclasses
    protected void breathe() { System.out.println('Breathing'); }
    private void heartBeat() { /* private - not accessible */ }
}

package com.company.animals; // different package
import com.company.base.Animal;
public class Dog extends Animal {
    public void bark() {
        super.breathe(); // ■ OK - protected, subclass
        System.out.println(species); // ■ OK - protected field
    }
}

package com.company.main;
Animal animal = new Animal();

```

```
// animal.breathe(); // ■ ERROR - different package, not a subclass
```

✓ Protected creates a 'semi-public' API for inheritance hierarchies. Pitfall: Protected fields break encapsulation in the inheritance chain — changes to the field affect all subclasses.

■ **Warning:** Effective Java (Joshua Bloch): 'Design and document for inheritance or else prohibit it.' Protected methods become part of your public API for subclasses — hard to remove later.

■ Director-Level Follow-up Questions:

- What does Joshua Bloch mean by 'design for inheritance or prohibit it'?
- How does 'protected' in Java differ from 'protected' in C++?

Q8. Explain types of nested classes in Java. When would you use each?

```
// 1. STATIC NESTED CLASS — associated with OUTER class, not instance
class Outer {
    static class StaticNested {
        void display() { System.out.println('Static Nested'); }
    }
}
Outer.StaticNested sn = new Outer.StaticNested(); // no Outer instance needed
// USE: Builder pattern — new Pizza.Builder().size('L').build()

// 2. INNER CLASS (non-static) — tied to OUTER instance, can access outer fields
class Outer {
    int x = 10;
    class Inner {
        void display() { System.out.println(x); } // accesses outer.x
    }
}
Outer outer = new Outer();
Outer.Inner inner = outer.new Inner(); // needs outer instance
// USE: Iterator pattern — LinkedList.ListItr

// 3. LOCAL CLASS — defined inside a METHOD
void processData() {
    class LocalHelper {
        void help() { System.out.println('Helping locally'); }
    }
    new LocalHelper().help();
    // Rarely used — usually replaced by lambdas
}
```

```
// 4. ANONYMOUS CLASS — no name, defined and instantiated in one shot
Runnable r = new Runnable() {
    @Override public void run() { System.out.println('Anonymous!'); }
};

// Replaced by lambdas: Runnable r = () -> System.out.println('Lambda!');
```

✓ Static Nested → utility/helper classes. Inner → event listeners, iterators. Anonymous → one-time callbacks (now replaced by lambdas).

■ **Director-Level Follow-up Questions:**

- *Why does an inner class hold a reference to the outer class? What memory leak can this cause?*
 - *How did anonymous classes evolve into lambdas? Are they exactly equivalent?*
-

Chapter 3: Java 8 → 21 Features

Lambdas, Streams, Records, Sealed Classes

Q9. Explain Functional Interfaces in depth. What are Predicate, Function, Consumer, Supplier?

Functional Interface: An interface with EXACTLY ONE abstract method. Can be implemented via lambda. @FunctionalInterface annotation is optional but enforced by compiler.

```
// The 4 core functional interfaces:

// 1. PREDICATE<T> – takes T, returns boolean (for filtering)
Predicate<String> isLong = s -> s.length() > 5;
Predicate<String> startsA = s -> s.startsWith('A');
Predicate<String> combined = isLong.and(startsA); // Predicate chaining!
System.out.println(combined.test('Alexander')); // true

// 2. FUNCTION<T, R> – takes T, returns R (for transformation)
Function<String, Integer> length = String::length;
Function<String, String> upper = String::toUpperCase;
Function<String, String> upperLen = upper.andThen(s -> s + ' len:' + s.length());
System.out.println(upperLen.apply('hello')); // HELLO len:5

// 3. CONSUMER<T> – takes T, returns void (for side effects)
Consumer<String> print = System.out::println;
Consumer<String> log = s -> System.err.println('[LOG] ' + s);
Consumer<String> both = print.andThen(log);
both.accept('Hello'); // prints to both stdout and stderr

// 4. SUPPLIER<T> – takes nothing, returns T (for lazy creation)
Supplier<Employee> empFactory = () -> new Employee('Default', 50000);
Supplier<List<String>> listFactory = ArrayList::new;
Employee e = empFactory.get(); // lazy – only creates when called

// BiFunction, BiPredicate, BiConsumer – 2-argument versions
BiFunction<Integer, Integer, Integer> multiply = (a, b) -> a * b;
```

```

System.out.println(multiply.apply(5, 6)); // 30

// UnaryOperator<T> = Function<T, T> (same input/output type)
UnaryOperator<String> trim = String::trim;

// BinaryOperator<T> = BiFunction<T, T, T>
BinaryOperator<Integer> sum = Integer::sum;

```

✓ Predicate=filter, Function=map/transform, Consumer=forEach, Supplier=get/create. These four cover 95% of stream operations.

■ Director-Level Follow-up Questions:

- *What is the difference between `Function.andThen()` and `Function.compose()`?*
- *How does method reference (`::`) work internally? Is it the same as a lambda?*
- *What is a `BiFunction`? Give a real-world use case for it in stream operations.*
- *Why can't a functional interface have two abstract methods? What about default methods?*

Q10. Deep Stream API — Explain intermediate vs terminal operations, lazy evaluation, and parallel streams.

Stream Pipeline: Source → Intermediate operations (lazy) → Terminal operation (triggers execution).

```

// Intermediate (lazy, return Stream):
// filter(), map(), flatMap(), distinct(), sorted(), peek(), limit(), skip()

// Terminal (eager, trigger execution):
// collect(), count(), findFirst(), findAny(), anyMatch(), allMatch(),
// reduce(), forEach(), toArray(), min(), max()

// LAZY EVALUATION — nothing runs until terminal operation!
Stream<Integer> stream = Stream.of(1, 2, 3, 4, 5)
    .filter(n -> { System.out.println('filter: ' + n); return n > 2; })
    .map(n -> { System.out.println('map: ' + n); return n * 10; });
// Output so far: NOTHING! No terminal op yet.
stream.findFirst(); // NOW filter and map run, but only until first match!

// SHORT-CIRCUIT: findFirst/anyMatch stop early — HUGE performance win!

// COMPLETE STREAM SOLUTIONS:

// 1. Total salary by department
Map<String, Double> salaryByDept = employees.stream()

```

```

.collect(Collectors.groupingBy(
Employee::getDepartment,
Collectors.summingDouble(Employee::getSalary)
));

// 2. Email domain count
Map<String, Long> domainCount = employees.stream()
.map(e -> e.getEmail().substring(e.getEmail().indexOf('@') + 1))
.collect(Collectors.groupingBy(d -> d, Collectors.counting()));

// 3. First repeating character
Optional<Character> firstRepeat = 'abcde'.chars()
.mapToObj(c -> (char) c)
.collect(Collectors.groupingBy(c -> c, LinkedHashMap::new, Collectors.counting()))
.entrySet().stream()
.filter(e -> e.getValue() > 1)
.map(Map.Entry::getKey)
.findFirst(); // Optional[a]

// 4. Duplicate words
List<String> duplicates = Arrays.stream('java is great java is'.split(' '))
.collect(Collectors.groupingBy(w -> w, Collectors.counting()))
.entrySet().stream()
.filter(e -> e.getValue() > 1L)
.map(Map.Entry::getKey)
.collect(Collectors.toList());

// 5. PARALLEL STREAMS – uses ForkJoinPool
long count = largeList.parallelStream()
.filter(e -> e.getSalary() > 50000)
.count();
// Use when: data > 10k elements, operations are CPU-intensive and stateless
// DON'T use for: small lists, I/O operations, order-sensitive ops

// 6. flatMap – flattens nested structures
List<List<Integer>> nested = Arrays.asList(asList(1,2), asList(3,4), asList(5));
List<Integer> flat = nested.stream()
.flatMap(Collection::stream) // flattens inner streams
.collect(Collectors.toList()); // [1,2,3,4,5]

```

```
// 7. reduce – custom aggregation
int product = IntStream.rangeClosed(1, 5)
    .reduce(1, (a, b) -> a * b); // 120 (5!)
```

✓ Streams are NOT data structures — they're computation pipelines. They don't store data, are consumed once, and evaluated lazily for maximum performance.

■ **Warning:** COMMON BUG: Reusing a stream after terminal operation throws `IllegalStateException`! Always create new stream from source.

■ Director-Level Follow-up Questions:

- What is the difference between `map()` and `flatMap()`? Give a JSON/nested list example.
- When should you use `parallelStream()`? What are its pitfalls with shared mutable state?
- What is the `Splitter` interface? How does it enable parallel processing?
- How does `collect(Collectors.groupingBy())` work internally?
- Explain `Collector` interface — what are `supplier`, `accumulator`, `combiner`, `finisher`?

Q11. Explain Records in Java 16+. How are they different from traditional immutable classes? What are their limitations?

```
// Traditional immutable class – VERBOSE:
public final class Point {
    private final int x, y;
    public Point(int x, int y) { this.x = x; this.y = y; }
    public int x() { return x; }
    public int y() { return y; }
    @Override public boolean equals(Object o) {
        if (!(o instanceof Point)) return false;
        Point p = (Point) o;
        return x == p.x && y == p.y;
    }
    @Override public int hashCode() { return Objects.hash(x, y); }
    @Override public String toString() { return 'Point[x=' + x + ', y=' + y + ']'; }
}

// RECORD – all of the above in 1 line!
public record Point(int x, int y) {}

Point p = new Point(3, 4);
System.out.println(p.x()); // 3 (getter named x(), NOT getX(!))
System.out.println(p); // Point[x=3, y=4]
new Point(3,4).equals(new Point(3,4)); // true
```

```
// Compact canonical constructor (add validation)
public record Employee(String name, double salary) {
    public Employee { // compact constructor – no () with params
        if (salary < 0) throw new IllegalArgumentException('Salary must be >= 0');
        name = name.strip(); // normalize
    }
}

// Custom methods allowed
public record Circle(double radius) {
    public double area() { return Math.PI * radius * radius; }
    public double perimeter() { return 2 * Math.PI * radius; }
}

// LIMITATIONS of Records:
// 1. Cannot extend other classes (implicitly extends java.lang.Record)
// 2. All fields are final – cannot be mutable
// 3. Cannot declare instance fields outside the header
// 4. Cannot be abstract
// 5. Cannot use @Transient for JPA (workaround: use @Column annotations)
```

✓ Records are perfect for DTOs, value objects, and data carriers. Not suitable for JPA entities (need mutable fields) or classes requiring inheritance.

■ Director-Level Follow-up Questions:

- *Can Records implement interfaces? Can they implement Serializable?*
- *How do Records interact with JPA/Hibernate? What are the limitations?*
- *What is the difference between a Record and a Kotlin data class?*

Q12. Explain Sealed Classes. What problems do they solve? How do they work with pattern matching?

```
// PROBLEM without sealed: Anyone can extend Shape – library author loses control
public abstract class Shape {} // ANY class in ANY package can extend!

// SEALED – restrict who can extend
public sealed class Shape permits Circle, Rectangle, Triangle {
    abstract double area();
}
```

```

public final class Circle extends Shape {
    public Circle(double radius) { this.radius = radius; }
    private final double radius;
    public double area() { return Math.PI * radius * radius; }
}

public final class Rectangle extends Shape {
    public Rectangle(double w, double h) { this.w=w; this.h=h; }
    private final double w, h;
    public double area() { return w * h; }
}

public non-sealed class Triangle extends Shape {
    // non-sealed: allows further extension
    private double base, height;
    public double area() { return 0.5 * base * height; }
}

// POWER: Pattern Matching in switch (Java 21)
String describe(Shape shape) {
    return switch (shape) {
        case Circle c -> 'Circle with radius ' + c.radius();
        case Rectangle r -> 'Rectangle ' + r.w() + 'x' + r.h();
        case Triangle t -> 'Triangle with base ' + t.base();
        // NO default needed – compiler knows ALL subtypes!
        // If you add a new subtype and forget to add case – COMPILE ERROR!
    };
}

// SEALED INTERFACES – great for Result types (Rust-style error handling)
sealed interface Result<T> permits Success, Failure {}
record Success<T>(T value) implements Result<T> {}
record Failure<T>(String error) implements Result<T> {}

Result<User> result = userService.findById(id);
String message = switch(result) {
    case Success<User> s -> 'Found: ' + s.value().getName();
    case Failure<User> f -> 'Error: ' + f.error();
};

```

✓ Sealed classes enable exhaustive type checking. Combined with Records and Pattern Matching, they implement algebraic data types (like Rust/Haskell) in Java.

■ **Note:** Java 21: Sealed + Records + Pattern Matching = full Algebraic Data Types. This is a fundamental shift in Java's type system.

■ **Director-Level Follow-up Questions:**

- *How do sealed classes relate to the Visitor pattern? Can sealed classes replace it?*
 - *Compare sealed classes in Java vs discriminated unions in TypeScript vs sum types in Scala.*
 - *How does sealed help with JSON deserialization for polymorphic types?*
-

Chapter 4: Collections Framework

Internal Working, Complexity, Thread Safety

Q13. Explain HashMap internal working — hashing, collision, treeification, and rehashing in depth.

Internal Structure: HashMap uses an array of Node[] (buckets). Default capacity=16, load factor=0.75.

```
// Node structure:
static class Node<K,V> {
    final int hash;
    final K key;
    V value;
    Node<K,V> next; // linked list for collision
}

// PUT operation – step by step:
// 1. hash = key.hashCode() XOR (hash >>> 16) // spread bits
// 2. index = hash AND (capacity - 1) // always in range [0, capacity-1]
// 3. If table[index] is null → new Node
// 4. If collision → add to LinkedList
// 5. If LinkedList size >= 8 AND capacity >= 64 → TREEIFY to Red-Black Tree
// 6. Red-Black Tree shrinks back to LinkedList when size <= 6
// 7. If size > capacity * 0.75 → REHASH (double capacity, re-index all nodes)

// WHY XOR with shift? Better hash distribution – uses BOTH halves of hash code
// WHY power-of-2 capacity? index = hash & (cap-1) is faster than hash % cap

// GET operation:
// 1. Compute hash for key
// 2. Find bucket index
// 3. Traverse LinkedList/TreeNode → compare with equals()
// 4. Return value if found, null otherwise

// REHASHING – expensive O(n) operation:
// Creates new array of 2x capacity
```

```
// Re-computes index for every existing entry
// Tip: if you know size, initialize with capacity to avoid rehashing:
Map<String, String> map = new HashMap<>(100); // avoids rehashing up to 75 entries

// RED-BLACK TREE (Java 8+):
// Worst case with LinkedList: O(n) for get/put with many collisions
// Red-Black Tree: worst case O(log n) – self-balancing BST
// Properties: each node red or black, no two consecutive reds, same black depth
```

✓ **HashMap:** average O(1) get/put. Worst case (before Java 8): O(n). After Java 8 with treeification: O(log n). Load factor 0.75 = 75% full before resize.

■ **Note:** NULL KEY: Only one null key allowed — stored at bucket[0] (hash is 0 for null). ConcurrentHashMap does NOT allow null keys/values.

■ Director-Level Follow-up Questions:

- What happens if hashCode() always returns the same value? What's the performance impact?
- Explain the Red-Black Tree properties. Why is it preferred over AVL Tree in HashMap?
- How does ConcurrentHashMap avoid synchronizing the entire map? What is segment-level locking?
- If you store a mutable object as a key in HashMap and change its state, what happens?
- How does Java 8's compute(), merge(), putIfAbsent() improve on traditional put()?

Q14. Explain HashSet vs LinkedHashSet vs TreeSet — internal working, time complexity, use cases.

```
// HashSet: backed by HashMap – key=element, value=PRESENT dummy object
HashSet<String> hs = new HashSet<>();
hs.add('banana'); // map.put('banana', PRESENT)
// add/contains/remove: O(1) average | No order guarantee
// NULL: allows one null

// LinkedHashSet: backed by LinkedHashMap – maintains INSERTION ORDER
// Extra: doubly-linked list connecting all entries
LinkedHashSet<String> lhs = new LinkedHashSet<>();
lhs.add('banana'); lhs.add('apple'); lhs.add('cherry');
// Iteration: banana → apple → cherry (insertion order)
// add/contains: O(1) | MORE memory than HashSet (linked list overhead)

// TreeSet: backed by TreeMap – RED-BLACK TREE – SORTED order
TreeSet<String> ts = new TreeSet<>();
ts.add('banana'); ts.add('apple'); ts.add('cherry');
// Iteration: apple → banana → cherry (natural/comparator order)
// add/contains/remove: O(log n) | NULL: NOT allowed!
```

```
// TreeSet navigation methods:
ts.first(); // 'apple'
ts.last(); // 'cherry'
ts.headSet('c'); // elements < 'c': [apple, banana]
ts.tailSet('b'); // elements >= 'b': [banana, cherry]
ts.subSet('a', 'c'); // 'a' <= x < 'c': [apple, banana]
ts.floor('b'); // <= 'b': 'banana'
ts.ceiling('b'); // >= 'b': 'banana'

// WHEN TO USE:
// HashSet: when you just need uniqueness, order doesn't matter
// LinkedHashSet: unique + maintain insertion order (e.g., recent searches)
// TreeSet: unique + sorted (e.g., leaderboards, word dictionaries)
```

✓ All three prevent duplicates. HashSet=fastest, LinkedHashSet=ordered, TreeSet=sorted but slowest. Choose based on: speed vs order vs sorted requirement.

■ Director-Level Follow-up Questions:

- How does TreeSet handle duplicates if you provide a Comparator that returns 0?
- Can you have a TreeSet of custom objects without implementing Comparable? What happens?
- EnumSet — when and why is it preferred over HashSet for enum collections?

Q15. Explain Comparable vs Comparator — when to use which? Show anonymous class, lambda, and method reference approaches.

```
// COMPARABLE: Natural/Default ordering — class implements it
// Use when: class has ONE obvious natural order
class Employee implements Comparable<Employee> {
    String name; int salary;
    @Override
    public int compareTo(Employee other) {
        return Integer.compare(this.salary, other.salary); // ascending
    }
}

List<Employee> list = new ArrayList<>(employees);
Collections.sort(list); // uses compareTo automatically

// COMPARATOR: Custom/External ordering — separate object/lambda
// Use when: multiple sort criteria OR can't modify the class
```

```
// 1. Anonymous Inner Class (Java 5 style)
Comparator<Employee> bySalaryDesc = new Comparator<Employee>() {
    @Override
    public int compare(Employee e1, Employee e2) {
        return Integer.compare(e2.salary, e1.salary); // reversed for desc
    }
};

// 2. Lambda (Java 8)
Comparator<Employee> byName = (e1, e2) -> e1.name.compareTo(e2.name);

// 3. Method Reference (most readable)
Comparator<Employee> bySalaryAsc = Comparator.comparing(Employee::getSalary);

// 4. CHAINED Comparator – sort by dept, then by salary desc
Comparator<Employee> multiSort = Comparator
    .comparing(Employee::getDepartment)
    .thenComparing(Comparator.comparing(Employee::getSalary).reversed());
list.sort(multiSort);

// RULE for compare/compareTo return value:
// negative = first comes BEFORE second
// zero = equal
// positive = first comes AFTER second
```

✓ Comparable: class has ONE natural order. Comparator: multiple sort criteria, or sorting external/third-party classes you can't modify.

■ Director-Level Follow-up Questions:

- What is the contract for `compareTo()`? What happens if it's inconsistent with `equals()`?
- How does `Comparator.naturalOrder()` work? What is `Comparator.nullsFirst()` and `nullsLast()`?
- In a `TreeMap`, if `compareTo()` returns 0 but `equals()` returns false, is the key overwritten?

Chapter 5: Design Patterns

Creational, Structural, Behavioral — Deep Implementation

Q16. Explain Singleton Pattern. What are all the ways to implement it? Which is the best and why?

```
// 1. EAGER INITIALIZATION – simple but wastes memory if never used
class Singleton1 {
    private static final Singleton1 INSTANCE = new Singleton1(); // created at class load
    private Singleton1() {}
    public static Singleton1 getInstance() { return INSTANCE; }
}

// 2. LAZY + SYNCHRONIZED – thread-safe but slow (every call acquires lock)
class Singleton2 {
    private static Singleton2 instance;
    private Singleton2() {}
    public static synchronized Singleton2 getInstance() {
        if (instance == null) instance = new Singleton2();
        return instance;
    }
}

// 3. DOUBLE-CHECKED LOCKING (DCL) – best of both worlds
class Singleton3 {
    private static volatile Singleton3 instance; // volatile prevents partial init!
    private Singleton3() {}
    public static Singleton3 getInstance() {
        if (instance == null) { // Check 1: no lock
            synchronized (Singleton3.class) {
                if (instance == null) { // Check 2: with lock
                    instance = new Singleton3();
                }
            }
        }
    }
}
```

```

return instance;
}
}

// 4. BILL PUGH / HOLDER PATTERN – lazy, thread-safe, no synchronization overhead
class Singleton4 {
    private Singleton4() {}
    private static class Holder {
        // Inner class loaded only when getInstance() is called
        private static final Singleton4 INSTANCE = new Singleton4();
    }
    public static Singleton4 getInstance() { return Holder.INSTANCE; }
}

// 5. ENUM SINGLETON (Joshua Bloch's recommendation) – BEST
public enum AppConfig {
    INSTANCE;
    private final String dbUrl = 'jdbc:mysql://localhost/db';
    public String getDbUrl() { return dbUrl; }
    // Handles serialization, reflection attacks automatically!
}
// Usage: AppConfig.INSTANCE.getDbUrl()

```

✓ Best: Enum Singleton. It's thread-safe, prevents reflection attacks, and handles serialization. For lazy loading: Bill Pugh (Holder) pattern.

■ **Warning:** DCL without 'volatile' is broken pre-Java 5 due to memory model issues. Reflection can break Singleton except with Enum.

■ Director-Level Follow-up Questions:

- How can Reflection break a Singleton? How does Enum prevent this?
- How does serialization break Singleton? What is readResolve()?
- How does Spring's @Bean(scope='singleton') differ from Singleton design pattern?
- Can a Singleton be thread-safe while its internal state is not?

Q17. Explain Factory vs Abstract Factory. What problem does each solve?

```

// FACTORY METHOD – one factory, one product type
interface Button { void render(); }
class WindowsButton implements Button { public void render() { System.out.println('Win Button'); } }
class MacButton implements Button { public void render() { System.out.println('Mac Button'); } }

```

```

class ButtonFactory {
public static Button create(String os) {
return switch(os) {
case 'WINDOWS' -> new WindowsButton();
case 'MAC' -> new MacButton();
default -> throw new IllegalArgumentException(os);
};
}
}

// ABSTRACT FACTORY – factory of factories – entire FAMILY of products
// Problem: GUI app needs CONSISTENT Windows OR Mac theme across all widgets

interface Button { void render(); }
interface Checkbox { void check(); }
interface Textfield { void type(); }

// Abstract Factory – creates FAMILY
interface GUIFactory {
Button createButton();
Checkbox createCheckbox();
Textfield createTextfield();
}

class WindowsFactory implements GUIFactory {
public Button createButton() { return new WinButton(); }
public Checkbox createCheckbox() { return new WinCheckbox(); }
public Textfield createTextfield(){ return new WinTextField(); }
}

class MacFactory implements GUIFactory {
public Button createButton() { return new MacButton(); }
public Checkbox createCheckbox() { return new MacCheckbox(); }
public Textfield createTextfield(){ return new MacTextField(); }
}

// Application uses factory – doesn't know which OS!
class Application {
private GUIFactory factory;
Application(GUIFactory factory) { this.factory = factory; }
}

```

```

void buildUI() {
    Button b = factory.createButton();
    b.render();
}
}

// new Application(new WindowsFactory()) – all widgets are consistent!

```

✓ Factory Method: one product, multiple implementations. Abstract Factory: family of related products that must be used together. Abstract Factory = 'family-level' OCP.

■ Director-Level Follow-up Questions:

- *How does Spring's ApplicationContext act as an Abstract Factory?*
- *When does a Factory Method become an Abstract Factory? What's the trigger?*
- *How does the Builder pattern differ from Factory? When would you choose one over the other?*

Q18. Explain Strategy vs Decorator vs Observer patterns with real-world Spring Boot examples.

```

// STRATEGY – swap algorithms at runtime
// Real-world: Payment processing, sorting algorithms, compression
interface SortStrategy<T> { List<T> sort(List<T> data); }
class QuickSort<T> implements SortStrategy<T> { ... }
class MergeSort<T> implements SortStrategy<T> { ... }
class DataProcessor<T> {
    private SortStrategy<T> strategy;
    void setStrategy(SortStrategy<T> s) { this.strategy = s; } // swap at runtime
    List<T> process(List<T> data) { return strategy.sort(data); }
}

// DECORATOR – add responsibilities without subclassing
// Real-world: Java I/O, Spring Security filters, HTTP interceptors
interface DataSource { void writeData(String data); String readData(); }
class FileDataSource implements DataSource { /* basic file I/O */ }
class EncryptionDecorator implements DataSource {
    private DataSource wrappee;
    EncryptionDecorator(DataSource ds) { this.wrappee = ds; }
    public void writeData(String data) { wrappee.writeData(encrypt(data)); }
    public String readData() { return decrypt(wrappee.readData()); }
}
class CompressionDecorator implements DataSource {
    private DataSource wrappee;

```

```

CompressionDecorator(DataSource ds) { this.wrappee = ds; }
public void writeData(String d) { wrappee.writeData(compress(d)); }
public String readData() { return decompress(wrappee.readData()); }
}

// Chain: new CompressionDecorator(new EncryptionDecorator(new FileDataSource()))
// Java I/O: new BufferedReader(new InputStreamReader(new FileInputStream('f.txt')))

// OBSERVER – publish-subscribe, event-driven
// Real-world: Spring Events, Message Queues, UI event listeners
@Component
class OrderService {
@Autowired ApplicationEventPublisher publisher;
void placeOrder(Order o) {
// business logic...
publisher.publishEvent(new OrderPlacedEvent(this, o)); // publish
}
}
@Component
class EmailNotificationListener {
@EventListener
public void handleOrder(OrderPlacedEvent event) {
// send email – decoupled from OrderService!
}
}

```

✓ Strategy=behavioral interchangeability, Decorator=structural layering, Observer=event-driven decoupling. Spring Boot uses all three: @Qualifier(Strategy), Filters(Decorator), @EventListener(Observer).

■ Director-Level Follow-up Questions:

- *How does the Chain of Responsibility differ from Decorator?*
- *Spring Security uses FilterChain — is this Decorator or Chain of Responsibility?*
- *How does @Async in Spring Boot relate to the Observer pattern?*

Chapter 6: JVM Internals

Memory Model, GC, ClassLoader, Threading

Q19. Explain JVM memory areas in detail — what goes in heap, stack, metaspace, and PC register?

```
// JVM RUNTIME DATA AREAS:

// 1. HEAP — shared by ALL threads
// Young Generation (Eden + S0 + S1)
// Old Generation (Tenured)
// - Stores: Object instances, arrays, instance variables
// - GC collected

// 2. STACK — one per THREAD, not shared
// - Stores: stack frames (method call info)
// - Each frame has: local variables, operand stack, frame data
// - Primitives and references (not objects) stored here
// - Automatically freed when method returns
// - StackOverflowError = too deep recursion

// 3. METASPACE (Java 8+, replaced PermGen)
// - Stores: class metadata, method bytecode, static variables, string pool
// - Grows dynamically (limited by OS, not JVM by default)
// - -XX:MaxMetaspaceSize=256m to limit it

// 4. PC (Program Counter) REGISTER — one per thread
// - Points to the currently executing JVM instruction

// 5. NATIVE METHOD STACK
// - For native (JNI) methods

// EXAMPLE:
void process() {
    int count = 5; // STACK: count=5
```

```
String name = 'Alice'; // STACK: name(ref) | HEAP/Pool: 'Alice'
Employee emp = new Employee(); // STACK: emp(ref) | HEAP: Employee object
emp.salary = 90000; // HEAP: salary field updated
} // method returns: stack frame POPPED, count and name refs gone

// STATIC vs INSTANCE FIELD STORAGE:
class Counter {
    static int total = 0; // METASPACE (class-level)
    int id = 0; // HEAP (per-instance)
    final int MAX = 100; // HEAP if instance final field
    static final int LIMIT = 1000; // METASPACE (compile-time constant may be inlined)
}
```

✓ Stack = thread-local, fast, auto-managed. Heap = shared, GC-managed, where all objects live. Metaspace = class structures, method code, static fields.

■ **Note:** String Pool is in the Heap (in Java 7+, moved from PermGen). `String.intern()` moves a string to the pool.

■ Director-Level Follow-up Questions:

- What is the difference between PermGen and Metaspace? Why was PermGen removed?
- What is Escape Analysis in JVM? How can JVM allocate objects on the stack?
- What is false sharing in CPU cache and how does it affect concurrent Java code?
- How does volatile prevent CPU cache inconsistency? What memory barrier does it insert?

Q20. Explain Garbage Collection — generational hypothesis, GC algorithms, and how to choose the right GC.

```
// GENERATIONAL HYPOTHESIS: Most objects die young.
// Young Gen (Eden + Survivors): where new objects go
// Old Gen (Tenured): long-lived objects promoted from Young

// GC PROCESS:
// 1. Object created in Eden
// 2. Eden full → Minor GC (fast, STW pause)
// - Live objects move to Survivor S0
// - Dead objects collected
// 3. Next Minor GC: S0 live → S1, age incremented
// 4. After 'tenuring threshold' (default 15): promote to Old Gen
// 5. Old Gen full → Major GC / Full GC (slow, long STW pause)

// GC ALGORITHMS:
// Serial GC: Single-threaded. Good for small apps, embedded systems.
```

```
// -XX:+UseSerialGC

// Parallel GC: Multi-threaded young gen GC. DEFAULT before Java 9.
// 'Throughput collector' – maximize throughput, tolerate pauses.
// -XX:+UseParallelGC

// G1 GC: Default in Java 9+. Region-based (not contiguous generations).
// Predictable pause time goals. Balances throughput & latency.
// -XX:+UseG1GC -XX:MaxGCPauseMillis=200

// ZGC: Java 15+ production. Pause < 10ms even with TB heaps!
// Concurrent – runs ALONGSIDE application threads.
// -XX:+UseZGC

// Shenandoah: Similar to ZGC, from RedHat. Concurrent compaction.
// -XX:+UseShenandoahGC

// WHEN TO USE:
// High throughput batch jobs → Parallel GC
// General server apps → G1GC (default, usually fine)
// Low-latency microservices, trading systems → ZGC or Shenandoah

// REFERENCE TYPES (affects GC behavior):
// Strong Reference: Object o = new Object(); // never GC'd while reachable
// Soft Reference: SoftReference<Cache> ref; // GC'd ONLY when memory low
// Weak Reference: WeakReference<obj> ref; // GC'd at NEXT GC cycle
// Phantom Reference: only after finalize(), used with ReferenceQueue
```

✓ Generational GC exploits the observation that most objects live and die quickly. G1 is default in Java 9+. For ultra-low latency: ZGC (Java 15+).

■ Director-Level Follow-up Questions:

- *What is a Stop-The-World (STW) pause? How does ZGC achieve sub-10ms pauses?*
- *What is a memory leak in Java if GC exists? Give 3 real examples.*
- *How do you analyze GC logs to diagnose production performance issues?*
- *What is the difference between a memory leak and a memory pressure issue?*

Q21. Explain ClassLoader hierarchy and custom ClassLoader. What is the delegation model?

```
// CLASSLOADER HIERARCHY:
```

```

// Bootstrap ClassLoader (C/C++) → loads core Java classes (java.lang.*)
// ↑ parent
// Extension ClassLoader → loads lib/ext or javax.*
// ↑ parent
// Application ClassLoader → loads classpath classes (your code!)
// ↑ parent
// Custom ClassLoader → your custom loader

// PARENT DELEGATION MODEL:
// When a class needs loading:
// 1. Child delegates to parent FIRST
// 2. Parent tries to load → success: use parent's version
// 3. Parent fails → child loads it
// This PREVENTS malicious code from replacing core Java classes!

// ClassNotFoundException vs NoClassDefFoundError:
// ClassNotFoundException: class not found at RUNTIME (Class.forName())
// NoClassDefFoundError: class WAS found at compile time but missing at runtime

// CUSTOM CLASSLOADER – for plugins, hot reload, isolation
class PluginClassLoader extends ClassLoader {
    private String pluginDir;

    PluginClassLoader(String dir, ClassLoader parent) {
        super(parent);
        this.pluginDir = dir;
    }

    @Override
    protected Class<?> findClass(String name) throws ClassNotFoundException {
        try {
            byte[] classBytes = loadClassBytes(name);
            return defineClass(name, classBytes, 0, classBytes.length);
        } catch (IOException e) {
            throw new ClassNotFoundException(name, e);
        }
    }

    private byte[] loadClassBytes(String className) throws IOException {
        String path = pluginDir + '/' + className.replace('.', '/') + '.class';
    }
}

```

```
return Files.readAllBytes(Paths.get(path));
}
}

// Use: PluginClassLoader loader = new PluginClassLoader('/plugins',
getClass().getClassLoader());
// Class<?> pluginClass = loader.loadClass('com.plugin.MyPlugin');
```

✓ Parent delegation ensures security (can't override java.lang.String). Custom ClassLoaders enable plugin systems (OSGi, Tomcat's isolated webapps, hot code reload).

■ **Director-Level Follow-up Questions:**

- *How does Tomcat use ClassLoaders to isolate web applications?*
 - *What is Class unloading? When does the JVM unload a class?*
 - *How does Java 9's Module System (JPMS) change the ClassLoader hierarchy?*
-

Chapter 7: Exception Handling

Hierarchy, Propagation, Best Practices

Q22. Explain the complete exception hierarchy. Difference between checked, unchecked, and Error.

```
// FULL HIERARCHY:
// Throwable
// ■■■ Error (JVM-level, don't catch)
// ■ ■■■ OutOfMemoryError
// ■ ■■■ StackOverflowError
// ■ ■■■ VirtualMachineError
// ■■■ Exception
// ■■■ Checked (must handle at compile time)
// ■ ■■■ IOException
// ■ ■■■ SQLException
// ■ ■■■ ClassNotFoundException
// ■ ■■■ InterruptedException
// ■■■ RuntimeException (Unchecked – no forced handling)
// ■■■ NullPointerException
// ■■■ ArrayIndexOutOfBoundsException
// ■■■ NumberFormatException
// ■■■ IllegalArgumentException
// ■■■ IllegalStateException
// ■■■ ConcurrentModificationException

// CHECKED: compiler forces you to handle (try-catch or throws declaration)
// UNCHECKED: programmer's error, not forced to handle

// Exception Overriding rules:
class Parent {
void method() throws IOException {}
}
class Child extends Parent {
// ■ Can throw same or narrower checked exception
```

```
void method() throws FileNotFoundException {} // FileNotFoundException ⊂ IOException
// ■ Can throw no exception
// void method() {} // OK!
// ■ Can throw any unchecked exception
// void method() throws RuntimeException {} // OK!
// ■ CANNOT throw BROADER checked exception
// void method() throws Exception {} // COMPILE ERROR!
}
```

✓ Checked = recoverable situations (file not found, DB down). Unchecked = programming errors (null ptr, bad index). Error = JVM failures (out of memory).

■ **Note:** *Best Practice (Effective Java): Use unchecked exceptions for programming errors. Use checked exceptions for recoverable conditions only when the caller can reasonably handle it.*

■ **Director-Level Follow-up Questions:**

- *When should you create a checked exception vs unchecked exception?*
 - *Why does Spring wrap checked exceptions as unchecked `DataAccessException`?*
 - *What is exception chaining? What is the cause mechanism?*
 - *Explain try-with-resources — how does it work for multiple resources? What if both `resource.close()` and the body throw exceptions?*
-

Chapter 8: Multithreading

Concurrency, Thread Safety, Synchronization

Q23. Explain volatile vs synchronized vs AtomicInteger. When to use each?

```
// VISIBILITY PROBLEM – CPU caches can have stale values:
class StopFlag {
    boolean flag = false; // NOT volatile
}
// Thread-1 sets flag = true
// Thread-2 may NEVER see the change! (reading from CPU cache)

// VOLATILE – ensures visibility, NOT atomicity
class StopFlagSafe {
    volatile boolean flag = false;
    // Now every write goes to main memory, every read from main memory
}

// PROBLEM: volatile doesn't help with compound operations:
volatile int count = 0;
count++; // NOT ATOMIC! This is Read-Modify-Write:
// Thread1: read count=5
// Thread2: read count=5 ← race!
// Thread1: write count=6
// Thread2: write count=6 ← should be 7!

// SYNCHRONIZED – mutual exclusion (only ONE thread at a time)
class SynchronizedCounter {
    private int count = 0;
    public synchronized void increment() { count++; } // OK, but slow
    public synchronized int getCount() { return count; }
}

// ATOMICINTEGER – lock-free atomicity via CAS (Compare-And-Swap)
class AtomicCounter {
```

```

private AtomicInteger count = new AtomicInteger(0);
public void increment() { count.incrementAndGet(); } // atomic, lock-free!
public int getCount() { return count.get(); }
}

// CAS: if(current == expected) set(newValue) – done as single CPU instruction!

// PERFORMANCE: AtomicInteger >> synchronized for simple counters
// For complex compound operations: use synchronized or ReentrantLock

// ReentrantLock – more flexible than synchronized
ReentrantLock lock = new ReentrantLock();
lock.lock();
try {
    // critical section
} finally {
    lock.unlock(); // ALWAYS unlock in finally!
}

// Extra: tryLock(), lockInterruptibly(), fair lock (new ReentrantLock(true))

```

✓ volatile=visibility only. synchronized=visibility+atomicity+mutual exclusion. AtomicXxx=lock-free atomicity via CAS. Use: AtomicInteger for counters, synchronized for compound operations, volatile for flags.

■ Director-Level Follow-up Questions:

- *What is a happens-before relationship in Java Memory Model?*
- *What is a deadlock? How do you detect and prevent it?*
- *Explain ThreadLocal. What memory leak can it cause in thread pool environments?*
- *What is the difference between wait/notify and Condition.await/signal?*

Chapter 9: Git Workflow

Merge, Rebase, Squash, Stash, Pull Requests

Q24. Merge vs Rebase vs Squash — explain with diagrams and when to use each in a team.

```
// INITIAL STATE:
// main: A → B → C
// feature: A → B → D → E

// 1. MERGE — creates a merge commit, preserves full history
git checkout main && git merge feature
// Result: A → B → C → M (merge commit)
// ■
// D → E
// ■ Shows full history ■ Cluttered with merge commits
// USE: public/main branches, when audit trail is needed

// 2. REBASE — replays feature commits on top of main
git checkout feature && git rebase main
// Result: A → B → C → D' → E' (new commits, linear history)
// ■ Clean linear history ■ Changes commit history (SHA changes!)
// USE: private feature branches before merging
// NEVER rebase shared branches!

// 3. SQUASH MERGE — all feature commits become ONE commit on main
git checkout main && git merge --squash feature && git commit -m 'Feature: login'
// Result: A → B → C → S (single squashed commit)
// ■ Cleanest main history ■ Loses granular commit history
// USE: When feature branch has messy WIP commits

// INTERACTIVE REBASE — pick/squash/fixup/reword commits
git rebase -i HEAD~5 // edit last 5 commits
// pick 1234abc commit 1
// squash 5678def commit 2 ← squash into previous
```

```
// fixup 9abc012 commit 3 ← squash, discard message  
// reword def3456 commit 4 ← change message
```

✓ Merge=preserve history, Rebase=clean linear history, Squash=atomic feature commit. Most teams: squash merge to main, rebase for local cleanup.

■ **Director-Level Follow-up Questions:**

- *What is a fast-forward merge? When does Git automatically do it?*
 - *What is 'origin/HEAD detached'? How do you recover from a detached HEAD?*
 - *How do git cherry-pick, bisect, and blame help in production debugging?*
-

Chapter 10: String, Memory & Immutability

String Pool, Integer Cache, Immutable Classes

Q25. How many String objects are created? Explain String Pool, intern(), and why String is immutable.

```
// SCENARIO 1:
String s1 = 'Hello'; // 1 object in String Pool
String s2 = 'Hello'; // 0 objects – REUSES from Pool!
System.out.println(s1 == s2); // true (same reference)

// SCENARIO 2:
String s3 = new String('Hello'); // 2 objects: Pool + new Heap object
System.out.println(s1 == s3); // false (different references)
System.out.println(s1.equals(s3)); // true (same content)

// SCENARIO 3 – concatenation:
String s = 'Hello' + ' World'; // COMPILE-TIME: 1 object 'Hello World' in pool
String a = 'Hello'; String b = ' World';
String c = a + b; // RUNTIME: creates StringBuilder, then new Heap String

// intern() – manually add to pool / get pool reference
String s4 = new String('Java').intern(); // now points to pool
System.out.println(s4 == 'Java'); // true after intern()

// WHY IS STRING IMMUTABLE?
// 1. SECURITY: SQL injection prevention, class loading security
// String className = 'java.lang.String'; // can't change after passing to ClassLoader
// 2. THREAD SAFETY: no synchronization needed – immutable objects are inherently thread-safe
// 3. STRING POOL: only works if strings are immutable (shared safely)
// 4. HASHCODE CACHING: String caches hashCode (computed once, reused)

// String hashCode is cached:
```

```
// private int hash; // default 0
// public int hashCode() {
// if (hash == 0) hash = compute(); // compute once, cache forever
// return hash;
// }
```

✓ 'hello' literal = 1 pool object. new String('hello') = 2 objects (pool + heap). String is immutable for: security, thread-safety, pool sharing, and hashCode caching.

■ **Director-Level Follow-up Questions:**

- *What is String deduplication in G1 GC (-XX:+UseStringDeduplication)? How does it work?*
 - *Why is String.format() slow? When should you use StringBuilder vs String.format()?*
 - *What is CharSequence? Why do many Java APIs accept CharSequence instead of String?*
-

Chapter 11: JUnit 5 Testing

Annotations, Assertions, Mocking

Q26. Explain all JUnit 5 annotations and assertion methods. How do you test exceptions?

```
// LIFECYCLE ANNOTATIONS:
@Test // marks a test method
@BeforeEach // runs before EACH test (setup)
@AfterEach // runs after EACH test (cleanup)
@BeforeAll // runs ONCE before all tests (must be static)
@AfterAll // runs ONCE after all tests (must be static)
@Disabled('reason') // skips test
@DisplayName('...') // custom display name
@Nested // nested test class for grouping
@Tag('integration') // tag tests for filtering

// ASSERTION METHODS:
assertEquals(expected, actual);
assertEquals(expected, actual, 'failure message');
assertNotEquals(unexpected, actual);
assertTrue(condition);
assertFalse(condition);
assertNull(object);
assertNotNull(object);
assertThrows(ExceptionType.class, () -> { risky(); });
assertDoesNotThrow(() -> { safeMethod(); });
assertAll( // all must pass
() -> assertEquals(1, calc.add(0,1)),
() -> assertEquals(4, calc.add(2,2)),
() -> assertTrue(calc.add(1,1) > 0)
);

// PARAMETERIZED TEST – test with multiple inputs
@ParameterizedTest
```

```

@ValueSource(strings = {'hello', 'world', 'java'})
void testLength(String input) { assertTrue(input.length() > 0); }

@ParameterizedTest
@CsvSource({'1, 2, 3', '5, 5, 10', '100, 200, 300'})
void testAdd(int a, int b, int expected) {
    assertEquals(expected, calculator.add(a, b));
}

// EXCEPTION TESTING – full example
@Test
void testParseInvalidInput() {
    NumberFormatException ex = assertThrows(
        NumberFormatException.class,
        () -> Integer.parseInt('not-a-number')
    );
    assertTrue(ex.getMessage().contains('not-a-number')); // verify message too!
}

// MOCK with Mockito
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock OrderRepository mockRepo;
    @InjectMocks OrderService orderService;

    @Test
    void placeOrderSuccess() {
        Order order = new Order(1L, 'Alice', 100.0);
        when(mockRepo.save(order)).thenReturn(order);
        Order result = orderService.placeOrder(order);
        assertEquals(order.getId(), result.getId());
        verify(mockRepo, times(1)).save(order); // verify interaction
    }
}

```

✓ JUnit 5 = Jupiter engine. Use `@BeforeEach` for setup, `assertThrows()` for exceptions, `@ParameterizedTest` for data-driven tests. Combine with Mockito for unit testing service layers.

■ Director-Level Follow-up Questions:

- What is the difference between `@Mock`, `@Spy`, and `@InjectMocks` in Mockito?
- How do you test private methods? Should you test them directly?
- Explain test pyramid — unit, integration, E2E. What percentage of each is ideal?

► *How do you test Spring Boot REST controllers using MockMvc vs WebClient?*

Chapter 12: Spring Boot

Auto-Configuration, DI, Logging

Q27. What are the disadvantages of Spring Boot? When would you NOT use it?

Advantages (context): Auto-configuration, embedded server, starter dependencies, Actuator, easy testing.

Disadvantages (be honest in interviews):

```
// 1. LARGE MEMORY FOOTPRINT
// Spring context loads hundreds of beans at startup
// A simple app uses 200-300MB RAM minimum
// Alternatives for lightweight: Micronaut, Quarkus (GraalVM native image)

// 2. SLOW STARTUP TIME
// Reflection-based DI (scanning classpath, creating proxies)
// 'Cold start' problem in serverless (AWS Lambda, Cloud Functions)
// Fix: Spring Native (GraalVM), lazy initialization, @Lazy

// 3. MAGIC / OPAQUE AUTO-CONFIGURATION
// Hard to understand WHY something is configured
// Debugging: spring.autoconfigure.exclude, condition evaluation report
// application.properties: debug=true → shows auto-configuration report

// 4. FAT JAR SIZE
// Includes embedded Tomcat + all dependencies
// Typical: 50-200MB jar

// 5. OVER-ENGINEERING FOR SIMPLE APPS
// For a simple script/utility: pure Java or Quarkus is better

// @Slf4j (Lombok) – generates:
private static final org.slf4j.Logger log =
    org.slf4j.LoggerFactory.getLogger(MyClass.class);

// LOG LEVELS: TRACE < DEBUG < INFO < WARN < ERROR
log.info('Processing order: {}'.format(orderId)); // parametrized (no string concat!)
```

```
log.error('Failed to process', exception); // logs stack trace
log.debug('Debug details: {}', details); // usually off in production
```

✓ Use Spring Boot for standard microservices. Avoid for: serverless (cold start), resource-constrained environments, simple scripts. Consider Quarkus/Micronaut for GraalVM native compilation.

■ **Director-Level Follow-up Questions:**

- *How does Spring Boot's auto-configuration work? What is @ConditionalOnClass?*
 - *Explain the Spring Boot startup sequence — from main() to first request handled.*
 - *How does Spring Boot Actuator help in production monitoring?*
 - *What is the difference between @Bean, @Component, @Service, @Repository in Spring?*
-

Chapter 13: Quick-Fire Q&A;

All Tricky Questions With Direct Answers

Q: Can you change a final object's field?

YES. `final` means the REFERENCE cannot be reassigned. The object itself is mutable.

```
final Employee e = new Employee();  
e.setName('Bob'); // ■ ALLOWED – changing field  
e = new Employee(); // ■ ERROR – can't reassign reference
```

Q: `System.out.println(1 < 2)` — what does it print and why?

Prints: true

1 and 2 are autoboxed to `Integer`. `Integer` implements `Comparable<Integer>`.
The `<` operator uses the primitive `int` comparison internally → returns boolean true.

Q: `String input = new String('Hari');`; `String output = 'Hari';` — how many objects?

3 total: 'Hari' in String Pool (compile time), new String object in Heap (runtime), output just references the Pool.

But `output = 'Hari'` creates 0 NEW objects (Pool already has 'Hari').

So: 2 new objects at runtime (1 pool + 1 heap from 'new String')

Q: `Integer a = 200; Integer b = 200; a == b?`

FALSE. Java caches `Integer` from -128 to 127 only.

200 > 127, so new `Integer` objects are created. `==` compares references, not values.

Use: `a.equals(b)` → TRUE

Q: What is hashCode contract?

1. Same object → same hashCode every time (within a JVM session)
2. `equals()==true` → MUST have same hashCode
3. `equals()==false` → SHOULD have different hashCode (not required, but better performance)

Q: Default GC in Java 9+?

G1GC (Garbage First). Java 11+ also defaults to G1.
Java 21: ZGC is now production-ready as alternative.

Q: HashMap vs TreeMap null handling?

HashMap: 1 null KEY allowed, multiple null VALUES allowed.
TreeMap: NULL KEY not allowed (throws NullPointerException – can't compare null for sorting).
ConcurrentHashMap: neither null keys NOR null values allowed.

Q: What is hybrid inheritance in Java?

Combination of multiple types (single + multiple + hierarchical).
Java doesn't support multiple CLASS inheritance (diamond problem).
Hybrid inheritance is achieved via INTERFACES (multiple interface implementation is allowed).

Q: What is a pure virtual function in Java?

Java doesn't use the term 'pure virtual'. The equivalent is an ABSTRACT METHOD in an abstract class.
`abstract void draw();` // pure virtual equivalent – must be implemented by subclasses

Q: What is a destructor in Java?

Java has NO destructor. Memory is managed by GC.
`finalize()` (deprecated Java 9+) was called by GC before collection – not reliable.
Alternative: Implement `AutoCloseable` and use `try-with-resources` for cleanup.

Q: Collections vs Stream?

Collections: in-memory data STORAGE. Can iterate multiple times. Eager evaluation.
Streams: computation PIPELINE over data. Single use. Lazy evaluation. No storage.
`Collections.stream()` creates a stream FROM a collection.

Q: YAGNI vs DRY vs KISS?

YAGNI (You Aren't Gonna Need It): don't code future features speculatively.
DRY (Don't Repeat Yourself): every piece of knowledge = ONE representation.

KISS (Keep It Simple, Stupid): prefer simple, readable code over clever code.

Q: What is diamond operator in Java?

The <> operator – type inference for generics (Java 7+).

```
Map<String, List<Integer>> map = new HashMap<>(); // <> infers type from declaration
```

Without: `new HashMap<String, List<Integer>>()` – redundant!

Q: throw vs throws?

throw: ACTUALLY throws an exception object: `throw new NullPointerException();`

throws: DECLARES in method signature that method may throw: `void m() throws IOException`

Q: final vs finalize()?

final: keyword – prevents reassignment (variable), overriding (method), subclassing (class).

finalize(): method on Object – called by GC before collection. DEPRECATED in Java 9. Unreliable. Don't use.

Chapter 14: Director-Level Generated Questions

Deep Architecture & System Design Questions

These questions go beyond implementation — they test your **SYSTEM THINKING** and **DESIGN JUDGMENT**.

■ System Design

- *Design a thread-safe LRU Cache in Java. What data structures? How do you handle concurrency?*
- *How would you design a distributed rate limiter using Java + Redis?*
- *Design an event-driven order processing system using Spring Boot + Kafka. How do you ensure exactly-once delivery?*
- *How do you choose between HashMap and ConcurrentHashMap in a microservice? What's the performance tradeoff?*
- *Design a custom connection pool in Java — what threading primitives would you use?*

■ JVM & Performance

- *Your Java application has a memory leak in production. Walk me through your investigation process.*
- *GC pauses are causing latency spikes. How do you diagnose and fix this?*
- *Explain how JIT (Just-In-Time) compilation works. What is tiered compilation?*
- *What is CPU profiling vs heap profiling? What tools (VisualVM, async-profiler, JFR) do you use?*
- *How does the Java Memory Model (JMM) relate to the CPU memory model? What is a memory barrier?*

■ Architecture & Patterns

- *When would you use Event Sourcing + CQRS over a traditional CRUD approach?*
- *Explain the Saga pattern for distributed transactions. Choreography vs Orchestration?*
- *How does the Circuit Breaker pattern (Resilience4j) help in microservice communication?*
- *When does Dependency Injection become over-engineering? How do you know when NOT to use it?*
- *Compare Hexagonal Architecture vs Layered Architecture for a large Spring Boot application.*

■ Java Advanced

- *Explain Project Loom (Virtual Threads, Java 21). How do they change the threading model?*

- *How does GraalVM native image work? What are the limitations (reflection, dynamic class loading)?*
- *Explain CompletableFuture — how does it compare to reactive programming (Project Reactor)?*
- *What is Structured Concurrency (Java 21)? How does it improve on traditional ExecutorService?*
- *How does pattern matching (Java 21) change how you write null-safe, type-safe code?*

■ Git & DevOps

- *Your team has a merge conflict in a critical production file. How do you resolve it?*
- *Explain Git branching strategies: GitFlow vs GitHub Flow vs Trunk-Based Development. Which do you recommend?*
- *How do you manage Maven/Gradle dependency versions across 50+ microservices?*
- *Explain how you would implement blue-green deployment using Spring Boot + Kubernetes.*
- *What is the difference between git reset --hard, --soft, and --mixed? When do you use each?*

Understand concepts deeply. Relate to real projects. Stay calm.